High-Level Design — ESP32 (Arduino) → BLE (NUS) → btCentral.py → CSV + UDP → sensordata_plotter.py

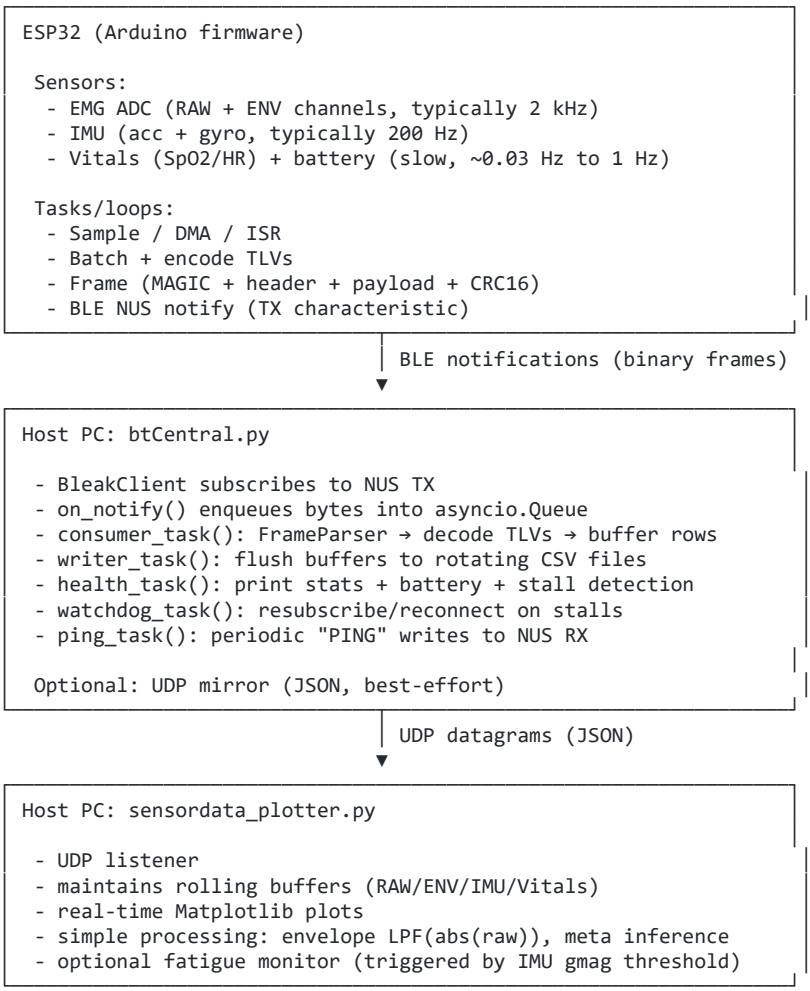
This document explains the end-to-end design of the system:

1. **ESP32 (Arduino firmware)** samples sensors (EMG, IMU, vitals/battery), packages data into a **binary framed protocol**, and streams it over **Bluetooth Low Energy** using the **Nordic UART Service (NUS)**.
2. **btCentral.py** runs on the host PC, connects as a BLE central, **parses frames**, **logs to rotating CSV files**, and optionally **mirrors selected data over UDP** as JSON for real-time visualization.
3. **sensordata_plotter.py** receives the UDP JSON stream and provides **real-time plots** plus some lightweight signal processing (LPF envelope, optional fatigue metrics).

   Paths/placement: This repository currently lives in `C:\wangx\ftc\arduino\py_scripts\`. Put this file in the **same folder** as `btCentral.py` and `sensordata_plotter.py`.

---

1) System blocks (high level)



---

2) BLE transport choice: Nordic UART Service (NUS)

UUIDs used (must match ESP32 firmware)

- **Service**: `6E400001-B5A3-F393-E0A9-E50E24DCCA9E`
- **TX (Notify)** (device → host): `6E400003-B5A3-F393-E0A9-E50E24DCCA9E`
- **RX (Write)** (host → device): `6E400002-B5A3-F393-E0A9-E50E24DCCA9E`

Why NUS

- Simple "serial over BLE" abstraction.
- Widely supported by ESP32 Arduino BLE libraries.
- Host can use **Bleak** cross-platform.

---

3) Binary frame protocol (ESP32 → host)

Frame structure

Each BLE notification carries a byte stream that the host reassembles into frames.

Header fields (little-endian):

- Magic: `0xA5 0x5A`
- `ver` (u8)
- `flags` (u8)
- `frame_seq` (u32)
- `t_us_u32` device timestamp in microseconds (u32, wraps)
- `payload_len` (u16)
- Payload: **TLV** list
- Optional CRC16-CCITT (u16) if `flags & FLAG_CRC16`

`btCentral.py` constants:

- `MAGIC = b"\xA5\x5A"`
- `HEADER_LEN = 14`, `CRC_LEN = 2`
- `FLAG_CRC16 = 0x01`

TLVs (Type-Length-Value)

Payload is a list of TLVs:

- `TLV_EMG_BATCH = 0x01`
- `TLV_IMU_BATCH = 0x02`
- `TLV_VITALS = 0x03`

Each TLV:

- `type` (u8)
- `len` (u16)
- `value` (len bytes)

Timebase strategy

- ESP32 sends `t_us_u32` (wraps every ~71.6 minutes).
- Host uses `DeviceClockExtender` to extend to a monotonic 64-bit `dev_us`.
- Each batch contains `t0_off` (offset from frame time), so the host reconstructs per-sample timestamps:
- `t0_us = dev_frame_us + t0_off`
- `sample_i_us = t0_us + i * (1_000_000 / sr_hz)`

This lets the device correct for buffering/DMA delays by choosing a better `t0_off`.

---

4) ESP32 (Arduino) side responsibilities

Even though the Arduino code is not shown here, `btCentral.py` implies the firmware must do the following.

4.1 Sensor capture

Typical approach:

- **EMG**
- Use ADC + DMA (preferred) or tight polling loop.
- Sample rate typically **2000 Hz**.
- Provide channels in a stable order (this Python assumes `samples[i,0]=env`, `samples[i,1]=raw`, optional `samples[i,2]=ref`).
- **IMU**
- Read accel + gyro, typically **200 Hz**.
- Provide raw int16 LSB values.
- Optionally include full-scale-range codes (FSR) in the IMU TLV meta (v2).
- **Vitals/Battery**
- Much slower updates.
- Include `age_ms` so host can compute when vitals were computed.
- Include `battery_pct` and `battery_mv` in the "new" format.

4.2 Batching

To reduce overhead, the firmware should batch multiple samples into one TLV:

- EMG batch: many samples per TLV
- IMU batch: many samples per TLV

This reduces BLE notification overhead and increases effective throughput.

4.3 Framing + CRC

Firmware must:

- Emit the MAGIC bytes so host can resync.
- Set `payload_len` correctly.
- If CRC is enabled (`FLAG_CRC16`), compute **CRC16-CCITT** over `header + payload` (excluding the CRC field itself).

4.4 BLE notify

- Use NUS TX notifications.
- Keep the packet size under MTU/notification limits (host requests MTU 247).
- Send continuously while connected.

4.5 Optional keepalive behavior

`btCentral.py` sends `PING` to NUS RX periodically.

On ESP32, typical behavior is:

- When a `PING` is received: update "last host seen" timestamp.
- If no PING for N seconds: optionally reduce sampling or stop streaming.
- If no PING for longer: optionally reboot BLE stack.

(Exact policy is firmware-defined; host supports it via `ping_task()`.)

---

5) btCentral.py internals (host recorder)

5.1 Major blocks

- **Bleak scanner + connect**
- Find device by `--device` address, or `--name`, or by NUS service UUID.
- **Notification callback → queue**
- `on_notify()` runs in a Bleak callback context.
- It uses `loop.call_soon_threadsafe(enqueue_chunk, bytes(data))`.
- `enqueue_chunk()` pushes bytes into an `asyncio.Queue`.
- **Frame parsing**
- `FrameParser.feed()` maintains a byte buffer and emits complete frames.
- Resynchronizes using MAGIC.
- Validates CRC (if enabled).
- **Decoding**
- `decode_emg()` converts TLV bytes → numpy array of u16.
- `decode_imu()` converts TLV bytes → numpy array of i16.
- `decode_vitals()` parses backward-compatible vitals (old and new).
- **CSV logging**
- EMG and IMU written via `TimeRotatingCsv` (time rotation).
- Vitals appended to `vitals_log.csv`.
- **UDP mirroring (optional)**
- If `--udp-enable`, `udp_send()` sends JSON packets via non-blocking UDP socket.
- **Never blocks recording**; failures are ignored.

5.2 Concurrency model

Async tasks:

- `consumer_task()` (decode + buffer) — driven by queue.
- `writer_task()` (flush to disk) — periodic.
- `health_task()` (print stats + warnings) — periodic.
- `ping_task()` — periodic BLE write.
- `watchdog_task()` — periodic stall monitor.

Key design choice: **BLE callback does minimal work** (just enqueues bytes). Parsing and IO happen in asyncio tasks.

5.3 Rotation / files

Session concept (one output folder = one session)

- A single run of `btCentral.py` is treated as **one session**.
- All files produced under `--outdir` during that run belong to that session.
- Files are **split into time segments** to keep each file manageable and reduce risk from partial corruption.
- Later/offline analysis should **stitch segments in order** (e.g., `..._000.csv`, `..._001.csv`, `..._002.csv`, ...).

Segment length (example: split every 20 minutes)

- `--rotate-min` controls how often EMG/IMU files rotate.

- For "split every 20 minutes", run with: `shell python btCentral.py --name MyoW1 --outdir logs --rotate-min 20`

What files are created

- EMG (rotating):
- `emg_log_<session_id>_<open_id>_<idx>.csv`
- `emg_log_<session_id>_<open_id>_<idx>.meta.json`
- IMU (rotating):
- `imu_log_<session_id>_<open_id>_<idx>.csv`
- `imu_log_<session_id>_<open_id>_<idx>.meta.json`
- Vitals (append-only in current implementation):
- `vitals_log.csv`

Rotating filename fields (how to interpret)

Example:

- `emg_log_20260226_135901_20260226_135901_000.csv`

Fields:

- `emg_log / imu_log`: stream type
- `<session_id>`: the session identifier created at process start (`time.strftime("%Y%m%d_%H%M%S")`)
- `<open_id>`: timestamp when the current segment file was opened (usually equals session_id for segment `000`)
- `<idx>`: 3-digit segment index starting at `000` and incrementing on each rotation

Why rotate

- Keeps files smaller for easier copying/sharing.
- Limits blast-radius if a file is partially written when power is lost.
- Enables "process-as-you-record" workflows where analysis can start on earlier segments.

---

6) UDP stream design (btCentral.py → sensordata_plotter.py)

Why UDP

- Real-time display without coupling plotter to BLE parsing.
- Best-effort transport is fine for visualization.
- Recorder is the source of truth (CSV logs).

Packet types (JSON)

`sensordata_plotter.py` expects JSON packets with field `k`:

- EMG meta:
- `{ "k":"emg_meta", "sr_hz":..., "ch":..., "adc_bits":..., "ymax":..., "seg":... }`
- EMG raw samples (decimated):
- `{ "k":"raw", "dev_us":..., "t":<host_ms>, "raw":..., "seg":... }`
- EMG env samples (decimated):
- `{ "k":"env", "dev_us":..., "t":<host_ms>, "env":..., "seg":... }`
- IMU magnitude (decimated):
- `{ "k":"imu", "dev_us":..., "t":<host_ms>, "amag":..., "gmag":..., "seg":... }`
- Vitals:
- `{ "k":"vitals", "pct":..., "v":..., "spo2":..., "hr":..., "seg":... }`

Decimation strategy

`btCentral.py` decimates to reduce UDP rate:

- RAW target: `--udp-raw-hz` (default 500 Hz)
- ENV target: `--udp-env-hz` (default 50 Hz)
- IMU target: `--udp-imu-hz` (default 50 Hz)

It computes:

- `emg_raw_decim = sr_hz // udp_raw_hz`
- `emg_env_decim = sr_hz // udp_env_hz`
- `imu_decim = sr_hz // udp_imu_hz`

and only sends when `(seq % decim) == 0`.

---

7) sensordata_plotter.py (real-time visualization + simple processing)

7.1 Live mode

- Binds UDP socket at `--listen-host/--listen-port`.
- Non-blocking `recvfrom()`; drains all pending packets each animation frame.
- Uses **device time** (`dev_us`) converted to `t_sec` relative to first packet.

7.2 Envelope processing

Plotter shows 3 curves:

- RAW (from UDP)
- ENV_device (from UDP)
- ENV_lpf(abs(RAW)) computed locally:
- causal 2nd order Butterworth LPF (default cutoff 3.6 Hz)
- this provides a sanity check against firmware envelope.

7.3 Sample-rate meta handling

- If it receives `emg_meta`, it configures filters and timebase.
- If the plotter starts late and misses `emg_meta`, it can infer `sr_hz` from dev_us deltas (`--infer-sr`).

7.4 Optional fatigue monitor

If enabled:

- Maintains a ring buffer of raw EMG samples.
- Uses IMU gyro magnitude threshold (`--imu-thr-dps`) to trigger a "swing".
- Calls `FatigueMonitor.push_swing()` in a background thread.
- Displays MDF trend and fatigue level.

---

8) Command line reference

8.1 btCentral.py (BLE recorder + CSV + optional UDP)

Synopsis

```
python btCentral.py [options]
```

Options (from argparse)

- `--outdir <path>`
- Output directory for logs (default: `logs`)
- `--device <ble_address>`
- BLE device address (if you want to connect directly without scanning by name)

- `--name <advertised_name>`
- BLE advertised name to match (e.g. `MyoW1`)
- `--scan-timeout <seconds>`
- Scan timeout (default: `10.0`)
- `--rotate-min <minutes>`
- Time-based rotation for EMG/IMU CSV files (default: `30.0`)
- `--queue-max <int>`
- Max number of queued BLE chunks before dropping (default: `20000`)
- `--flush-sec <seconds>`
- Flush buffers to disk interval (default: `2.0`)
- `--timeout <seconds>`
- BLE client connect/operation timeout (default: `60.0`)
- `--address-type {public,random}`
- Windows WinRT address type override (default: `None`)

Watchdog / keepalive

- `--ping-sec <seconds>`
- Period between host → device `PING` writes (default: `1.0`)
- `--stall-warn-ms <ms>`
- Health status becomes WARN when no notifications for this long (default: `2000`)
- `--stall-reconnect-ms <ms>`
- Trigger resubscribe + reconnect logic when no notifications for this long (default: `5000`)
- `--max-backoff-sec <seconds>`
- Exponential reconnect backoff cap (default: `20.0`)

UDP (optional real-time stream to plotter)

- `--udp-enable`
- Enable UDP JSON mirroring
- `--udp-host <host>`
- UDP target host (default: `127.0.0.1`)
- `--udp-port <port>`
- UDP target port (default: `9999`)
- `--udp-raw-hz <hz>`
- Target RAW UDP sample rate after decimation (default: `500`)
- `--udp-env-hz <hz>`
- Target ENV UDP sample rate after decimation (default: `50`)
- `--udp-imu-hz <hz>`
- Target IMU UDP sample rate after decimation (default: `50`)

8.2 sensordata_plotter.py (UDP plotter: live + offline)

Synopsis

```
python sensordata_plotter.py --mode live [options]
python sensordata_plotter.py --mode offline --emg-file <path> [options]
```

Options (from argparse)

- `--mode {live,offline}`
- Run mode (default: `live`)
- `--listen-host <host>`
- UDP bind host (default: `127.0.0.1`)
- `--listen-port <port>`
- UDP bind port (default: `9999`)

Display / performance

- `--emg-window-sec <seconds>`
- Rolling EMG window displayed (default: `2.0`)
- `--imu-window-sec <seconds>`
- Rolling IMU window (default: `None` → uses emg-window-sec)
- `--fps <fps>`

- Plot update FPS (default: `30.0`)
- `--emg-ymax <float>`
- EMG y-axis max (default: `4095.0`)
- `--cutoff-hz <float>`
- Cutoff for plotter's LPF envelope of abs(raw) (default: `3.6`)

Meta inference (if plotter starts after recorder and misses emg_meta)

- `--infer-sr`
- Infer EMG sample rate from `dev_us` deltas
- `--infer-min-hz <float>` (default: `100.0`)
- `--infer-max-hz <float>` (default: `4000.0`)
- `--infer-min-samples <int>` (default: `30`)

Offline mode

- `--emg-file <path>`
- CSV file to load (required for offline mode)
- `--max-points <int>`
- Max samples to load from file (default: `400000`)
- `--sr-hz <float>`
- Sample rate to assume for offline LPF processing (default: `2000.0`)

Fatigue monitoring (optional)

- `--fatigue`
- Enable fatigue monitor
- `--emg-util-path <dir>`
- Directory containing `emg_util.py` (default: `../../emg_util` relative to plotter)
- `--imu-thr-dps <float>`
- Trigger threshold for gyro magnitude (default: `150.0`)
- `--fatigue-warmup-min <float>`
- Warmup skip time before baseline collection (default: `0.0`)
- `--fatigue-baseline-swings <int>`
- Number of swings to collect for baseline (default: `20`)

---

9) Typical usage

Record only (CSV)

```
python btCentral.py --name MyoW1 --outdir logs --rotate-min 20
```

Record + UDP stream

```
python btCentral.py --name MyoW1 --outdir logs --rotate-min 20 --udp-enable --udp-port 9999
```

Plot (live)

```
python sensordata_plotter.py --mode live --listen-port 9999
```

Plot (live, infer sample rate if started late)

```
python sensordata_plotter.py --mode live --listen-port 9999 --infer-sr
```

---

10) Debug/validation checklist

BLE layer

- ESP32 advertises the NUS service UUID.
- Host can discover by name or by service.
- MTU request may or may not succeed; system should still work.

Protocol correctness

- MAGIC bytes present.
- `payload_len` matches actual payload.
- CRC16 matches host implementation when enabled.
- TLV lengths are correct (host will drop malformed TLVs).

Plotter correctness

- Verify RAW looks "high frequency" and ENV is slow-varying.
- If RAW and ENV look identical, check UDP decimation rates.

---

11) Swing detection algorithms

The system implements **two different swing detection algorithms** for different purposes:

11.1 Real-time gyroscope-based detection (simple, fast)

Used by:

- **Android app**: `SessionCaptureActivity.java` (`C:\wangx\ftc\arduino\btLeApp\app\src\main\java\com\clj\blesample\capture\SessionCaptureActivity.java`)
- **Python plotter**: `sensordata_plotter.py` (this directory)

Purpose: Real-time swing counting and fatigue monitoring during live sessions.

Algorithm: Peak detection on gyroscope magnitude

```
1. Calculate gyro magnitude: gmag = sqrt(gx² + gy² + gz²)
2. Convert to degrees per second: gmag_dps = gmag / GYRO_SCALE_FACTOR
3. Peak detection state machine:
   - When gmag_dps >= threshold (default 150 dps):
     * If not in peak: Start tracking new peak
     * If in peak: Update peak if current value is higher
   - When gmag_dps < threshold and in peak:
     * Trigger detection at the PEAK point (not at threshold crossing)
     * Apply 1-second debounce to prevent re-triggers
     * Increment swing counter
     * Queue EMG snapshot for fatigue analysis
```

Key parameters:

- Threshold: 150 dps (adjustable via `--imu-thr-dps` in plotter, SeekBar in Android)
- Debounce: 1 second (prevents double-counting backswing + forward swing)
- Scale factor: 16.4 LSB/dps (ICM-20948 at ±2000 dps range)

Why gyroscope (not accelerometer):

- Gyroscope measures **rotation rate** → directly captures swing motion
- Accelerometer measures **linear acceleration** → affected by gravity, arm movement, walking
- Tennis swings are primarily **rotational** movements
- Gyroscope provides cleaner, more reliable swing detection

Why peak detection (not simple threshold):

- Tennis swings have backswing + forward swing, both exceeding threshold

- Simple threshold would trigger twice per swing
- Peak detection finds the HIGHEST rotation point → one trigger per swing
- More accurate timing for EMG snapshot (captured at maximum effort)

Implementation consistency: Both Android (Java) and Python implementations use **identical logic**:

- Same sensor (gyroscope)
- Same threshold (150 dps default)
- Same debounce (1 second)
- Same peak tracking algorithm
- Same trigger timing (at peak, not threshold crossing)

This ensures consistent swing counts across real-time tools.

11.2 Offline EMG-based detection (sophisticated, accurate)

Used by:

- **Offline analysis**: `emg_util.py` in `C:\wangx\ftc\emg_util\`

Purpose: Post-session analysis with higher accuracy for research/training insights.

Algorithm: Multi-stage EMG envelope analysis

```
1. Bandpass filter EMG (20-450 Hz) to remove noise and motion artifacts
2. Compute envelope via Hilbert transform
3. Smooth envelope with moving average
4. Find peaks in envelope that exceed adaptive threshold
5. Validate peaks:
   - Minimum prominence (relative to surrounding baseline)
   - Minimum distance between peaks (prevents double-counting)
   - Minimum duration above threshold
6. Refine peak timing using local maxima search
```

Why EMG (not IMU) for offline:

- **EMG directly measures muscle activation** → ground truth for effort
- **IMU can miss swings** if rotation is below threshold (e.g., soft volleys, drop shots)
- **EMG captures all muscle contractions** regardless of swing speed/amplitude
- **Better for fatigue analysis** since MDF is computed from EMG anyway

Why more sophisticated:

- Offline analysis has **no real-time constraints** → can use complex algorithms
- Can **look ahead and behind** in time → better peak validation
- Can use **adaptive thresholds** based on session statistics
- Can **filter out false positives** (e.g., practice swings, adjustments)

Trade-offs:

Feature	Real-time (Gyro)	Offline (EMG)
Latency	<10ms	N/A (post-session)
Accuracy	~95%	~98%
False positives	Walking, gestures	Rare
False negatives	Soft swings	Very rare
Computational cost	Very low	Moderate
Use case	Live feedback	Research analysis

When to use which:

- **Real-time gyro**: Live sessions, immediate feedback, fatigue monitoring during play
- **Offline EMG**: Post-session analysis, training insights, research studies, ground truth validation

---

12) Notes on repository docs

- `PROTOCOL.md` contains a deeper protocol/architecture specification and is the authoritative reference for fields.
- This document is intended as the quick "how the pieces fit together" explanation.
- For swing detection implementation details, see:
- Real-time: `SessionCaptureActivity.java` (Android) and `sensordata_plotter.py` (Python)
- Offline: `emg_util.py` in the `emg_util/` directory